Copyright © 2023. Bentham Science Publishers. All rights reserved.

Sequence-String and List

Krishna Kumar Mohbey[1], **Malika Acharya**[1]

[1] Computer Science, Central University of Rajasthan, India

Abstract

Unlike primitive data types like integers, floats and Boolean, a string is an ordered sequence of characters each of which can be accessed easily. Further, one of the important built-in data types of Python is lists. Lists and strings share many similarities like, lists are a sequence of values. List indices work similarly to string indices. But unlike strings lists are mutable. In this chapter, we introduce the core concepts of lists and strings and several operators that are used to make programming with these user-friendly.

Keywords: Mutable, Slicing, Ordered collection.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

INTRODUCTION

When we want a collection of characters that are pretty like one another, we have to use a sequence of characters. For instance, if you want to keep a record of your name in the computer's memory, you will need a variable capable of storing your name. However, it is necessary to have a series of characters because the name is a collection of characters. This string consists of nothing more than a succession of characters.

Python's equivalent of the sequence data type is called a list. It is the most effective and may be expressed as a list of values delimited by commas and enclosed in square brackets. A list is frequently utilized to store the sequence of various kinds of data. The list is changeable, which indicates that its elements can be altered after the list has been constructed. This chapter discusses a variety of operations that may be carried out on a list's elements.

STRING

A series of characters is referred to as a string. Python relies heavily on this idea to function correctly. There are a few key aspects to consider about string.

- Strings are amongst the most popular types in Python.
- It can create them simply by enclosing characters in quotes (single, double, or triple).
- Python treats single quotes the same as double quotes.
- A string is a sequence of Unicode characters, and a character is simply a symbol.

To generate a list of names, it must type each name surrounded by quotation marks, such as “Krishna”. The string may also be assigned to a variable to carry out additional operations and make further use of that string. Let's start with the most fundamental examples to grasp the idea of string.

Example:

```
1.  str = "This is a string."
2.  print (str)
```

Output:

```
This is a string
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Python does not have a character data type in its standard library. If we consider a single character to be a string, then the length of that string would be one. You may access the string's constituent components using square brackets ([]).

Example:

```
1.  str = "Python"
2.  print (str[0])
3.  print (str[1])
4.  print (str[2])
5.  print (str[3])
6.  print (str[4])
7.  print (str[5])
```

Output:

```
P
y
t
h
o
n
```

The string can use to loop through character by character.

Example:

```
1.  for i in "Krishna":
2.    print (i)
```

Output:

```
K
r
i
s
h
n
a
```

Other concepts about string include the *len()* function for getting the length of a string, keyword *in*, and *not in* for checking whether the substring or character is present in a string.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
1.  # print a length of a string
2.  str = "Python"
3.  print (len(str))  #String Length
4.
5.  # print "simple" in a string
6.  str1= "Python is so simple"
7.  print ("simple" in str1)
8.
9.  #Print only if "simple" is present in a string:
10. str2 = "Python is so simple"
11. if "simple" in str2:
12.   print ("Yes, 'simple' is present.")
13.
14. # print "simple" is not in a string
15. str3 = "Python is so simple"
16. print ("programming" not in str3)
```

Output:

```
6
True
Yes, 'simple' is present.
True
```

String Concatenation

Combining two strings is what “string concatenation” refers to. If we have two strings placed into two different variables, str1 and str2, then the two strings may be concatenated together or merged into a single string by using one of those two methods. There are four ways to concatenate the strings.

1. Using + Operator
2. Using join() method
3. Using % operator
4. Using format() function

Using + Operator

It is a straightforward way to concatenate two strings. It can add multiple strings together. Let us take an example to understand this concept.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# Defining strings
S1 = "Python"
S2 = "Programming"

# + Operator is used to combine strings
S = S1 + S2
print (S)
```

Output:

```
PythonProgramming
```

Using join() Method

The join() method is a string method that returns a string in which the sequence elements have been joined using an str separator.

Example:

```
fname = "My"
lname = "Name"


print ("".join([fname, lname]))

#join() method string with a separator Space(" ")
name = " ".join([fname, lname])
  print (name)
```

Output:

```
MyName
MyName
```

Using % Operator

It can use a string formatting operator (%) for concatenating the strings.

Example:

```
fname = "My"
lname = "Name"

# % operator
print ("% s % s" % (fname, lname))
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Output:

```
MyName
```

Using Format() Function

This is an illustration of a way to format strings. It is possible to make many substitutes and format the value in multiple ways. *str.format()* is necessary to concatenate the strings. This approach achieves concatenating components within a series by using positional formatting. The following code demonstrates how to use the ***format()*** function to combine the strings that are now stored in the fname and lname variables into a single string that is then saved under a different variable name. It is possible to place strings in a specific sequence using curly

bracaes. The first variable is tracked in the first set of curly braces and second variable is traced in the second set of curly braces.

Example:

```
1.  fname = "Malika"
2.  lname = "Acharya"
3.    print ("{} {}".format(fname, lname))
4.
5.  # result in name
6.  name = "{} {}".format(fname, lname)
7.    print (name)
```

Output:

```
MalikaAcharya
MalikaAcharya
```

Appending Strings

Appending to a string means adding one string to another. Suppose we have two strings, and those strings have been assigned to two variables, **S1** and **S2**; then, it can be appended or added **S1** to **S2**. There are two ways to append one string to another.

1. Using += operator
2. Using join()

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Using += Operator

Utilizing a specialized function to complete this one activity is an example of the conventional method used in other languages, although this approach is simpler. To better grasp this idea, let's look at a concrete illustration.

Example:

```
# initializing string
str1 = "String1"
  # initializing add_string
str2 = "String2"
  # printing original string
print ("The original string : " + str(str1))
  # printing original add string
print ("The add string : " + str(str2))

# adding one string to another
str1 += str2          # Using += operator
  # print result
print ("The appended string is : " + str1)
```

Output:

```
The original string: String1
The add string: String2
The appended string is: String1String2
```

Using Join()

To join the strings together, this approach is utilized. The preceding technique has an advantage over this method when we need to concatenate more than just two strings, such as when we have multiple strings to concatenate. To better grasp this idea, let's look at a concrete illustration.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# initializing string
str1 = "Python"

# initializing add_string
str2 = "Book"
 # printing original string
print ("The original string : " + str(str1))
 # printing original add string
print ("The add string : " + str(str2))

# adding one string to another
result = "".join((str1, str2))          # Using join()
  # print result
print ("The appended string is : " + result)
```

Output:

```
The original string : Python
The add string : Book
The appended string is : PythonBook
```

Multiplying Strings

Because string multiplication in Python is analogous to the behavior of another data type, the process is much simplified. The process of multiplying strings can be done in several different ways. To better grasp this idea, let's look at a concrete illustration.

Example:

```
str1= "Python Programming"
str2= 2*str1    # Method 1
print (str2)

str3=4*(str1) # Method 2
print (str3)

str4= 3*('Python', 'Programming') # Method 3
print (str4)
```

Output:

```
Python ProgrammingPython Programming
Python ProgrammingPython ProgrammingPython ProgrammingPython Programming
('Python', 'Programming', 'Python', 'Programming', 'Python', 'Programming')
```

Immutable Strings

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

When a string is immutable, the value of the string cannot be altered in any way. In addition, once we have assigned the string value, an error will be generated if we try to perform this thing, and it cannot reassign itself. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  str= "Book"
2.  print (str)
3.  str[0] = "P"   # Can not reassign
```

Output:

```
str[0] = "V"   # Can not reassign
TypeError: 'str' object does not support item assignment
```

String Formatting Operator

It uses the % operator for string formatting operations. Various symbols can be used along with %, as shown in Table **1**.

Table 1 **String formatting symbols and their description.**

Symbol	Description
%f	It is for a floating-point real number
%d	It is for a signed decimal integer
%u	It is for an unsigned decimal integer
%o	It is for an octal integer
%X	It is for a hexadecimal integer (Uppercase letters)
%e	It is for an exponential notation (with lowercase 'e')
%E	It is for an exponential notation (with UPPERcase 'E')
%g	It is for the shorter of %f and %e
%G	It is for the shorter of %f and %E
%c	It is for a character
%s	It is for a string conversion *via* str() before formatting
%i	It is for a signed decimal integer

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

%x	It is for a hexadecimal integer (lowercase letters)

Example:

```
# Initialize the variable as a string
var = '27'
string = "Variable as string = %s" %(var)
print (string)

# Printing as raw data
print ("Variable as raw data = %r" %(var))

# Convert the variable to an integer
# and apply other formatting options
var = int(var)
string = "Variable as integer = %d" %(var)
print (string)
print ("Variable as float = %f" %(var))

print ("Variable as printing with special char = %c Ram" %(var))

print ("Variable as hexadecimal = %x" %(var))
print ("Variable as octal = %o" %(var))
```

Output:

```
Variable as string = 27
Variable as raw data = '27'
Variable as integer = 27
Variable as float = 27.000000
Variable as printing with special char = Ram
Variable as hexadecimal = 1b
Variable as octal = 33
```

Built-in String Functions

There are various types of built-in string functions. We use those functions and get the desired results. One crucial point is that these types of functions return the new string. It cannot change the original string. We are showing some essential built-in string functions as shown in Table **2**.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# built-in string functions
txt = "Python is a good prog., Python is easy to learn, Python is easy to understand."
cnt = txt.count("Python")
print (cnt)

low = txt.islower()  # check if all the characters in the text are in lowercase
print (low)

spl= txt.split()  #Split a string into a list where each word is a list item
print (spl)

ew = txt.endswith(".") #Check if the string ends with a (.)
print (ew)

cen = txt.center(10) # space of 10 characters
print (cen)

an = txt.isalnum()  #Check if all the characters in the text are alphanumeric
print (an)

isn = txt.isnumeric()  #Check if all the characters in the text are numeric
print (isn)

tit = txt.title() # first letter in each word upper case
print (tit)
```

Output:

```
3

False

['Python', 'is', 'a', 'good', 'prog.,', 'Python', 'is', 'easy', 'to', 'learn,', 'Python', 'is', 'easy', 'to', 'understand.']

True

Python is a good prog., Python is easy to learn, Python is easy to understand.

False

False

Python Is A Good Prog., Python Is Easy To Learn, Python Is Easy To Understand.
```

Table 2 **Built-in string functions.**

count()	It returns the number of times a specified value occurs in a string
islower()	It returns True if all characters in the string are lowercase
split()	It splits the string at the specified separator and returns a list
endswith()	It returns true if the string ends with the specified value

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

center()	It returns a centered string
isalnum()	It returns True if all characters in the string are alphanumeric
isnumeric()	It returns True if all characters in the string are numeric
title()	It converts the first character of each word to upper case

Slice Operation

The slice() function returns a slice object that can be used to slice strings, lists, tuples, *etc.* There is the following syntax with parameters description of slice() function.

Syntax:

```
slice(stop)
slice(start, stop, step)
```

A **start** parameter provides the index at which an object's slicing begins. The index at which the slicing of an object is stopped can be determined by using a **stop** argument. The **step** parameter is an optional statement that determines the increment between each index throughout the slicing process. There are the following important points about the *slice()* function:

- It returns a sliced object that only contains elements from the defined set.
- If only one parameter is passed, then the start and step are considered to be none.

Example:

```
# It starts the slice object at position 2,  slice to position 4, and return the result:
var = ("P", "Y", "T", "H", "O", "N")
sl = slice(2, 4)
print (var[sl])

#It use the step parameter to return every second item:
var = ("P", "Y", "T", "H", "O", "N")
sl = slice(0, 6, 2)
print (var[sl])
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
('T', 'H')
('P', 'T', 'O')
```

Example:

```
# String slicing
str ='Pythonbook'
sl1 = slice(2)
sl2 = slice(1, 4, 3)
print ("String slicing")
print (str[sl1])
print (str[sl2])

# List slicing
lst = [5, 6, 7, 8, 9]
sl1 = slice(2)
sl2 = slice(1, 4, 2)
print ("\nList slicing")
print (lst[sl1])
print (lst[sl2])

# Tuple slicing
tpl = (4, 5, 6, 7, 8)
sl1 = slice(3)
sl2 = slice(1, 5, 2)
print ("\nTuple slicing")
print (tpl[sl1])
print (tpl[sl2])
```

Output:

```
String slicing
Py
List slicing
[5, 6]
[6, 8]
Tuple slicing
(4, 5, 6)
(5, 7)
```

The ord() Function

It is a built-in feature that returns the integer value corresponding to a single Unicode character as an integer. The Unicode code point is represented by the number that was returned. When a string of length 1 is provided to the **ord()** function, it returns an integer representing the character's

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Unicode code point. If the string's length is larger than one, a TypeError will be generated. Let us look at an example better to comprehend the notion of ord() functions.

Example:

```
1.  res1 = ord('A')
2.  res2 = ord('a')
3.  print (res1, res2)   # prints the Unicode value
```

Output:

```
65 97
```

The chr() Function

The **chr()** function converts an integer into a character or string and returns the result. The value of the number is the Unicode code point for the character. To better comprehend the notion of **chr()** functions, let us look at an example.

Example:

```
1.  res1 = chr(65)   #return a character of Unicode valve 65
2.  res2 = chr(97)   #return a character of Unicode valve 97
3.  print (res1, res2)   # prints the character value
```

Output:

```
A a
```

String Comparison

Comparing strings means identifying whether the two strings are equivalent to each other or not. There are three ways to compare the strings:

1. Using Relational Operators
2. Using is and is not
3. Using user-defined function

Using Relational Operators

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

The relational operators compare the Unicode values of the characters of the strings. It returns a Boolean value according to the operator used. There are the following relational operators available in Python (Table **3**).

Table 3 **String comparison using relational operators.**

==	It checks whether two strings are equal.
!=	It checks if two strings are not equal.
<	It checks if the string on its left is smaller than on its right.
<=	It checks if the string on its left is smaller than or equal to that on its right.
>	It checks if the string on its left is greater than that on its right.
>=	It checks if the string on its left is greater than or equal to that on its right.

Example:

```
print ("Python" == "Python")
print ("Python" < "python")
print ("Python" > "python")
print ("Python" <= "python")
print ("Python" >= "python")
print ("Python" != "Python")
```

Output:

```
True
True
False
True
False
False
```

Using *is* and *is not*

This operator checks whether both operands refer to the same object or not. Let us take an example to understand this concept.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
str1 = "Book"
str2 = "Book"
str3 = str1

print (str1 is not str1)
print (str1 is not str2)
print (str1 is not  str3)

print (str1 is str1)
print (str1 is str2)
print (str1 is str3)
```

Output:

```
False
False
False
True
True
True
```

Using User-defined Function

We can make user-defined functions for string comparison. A user-defined function will compare the strings based on the number of digits.

Example:

```
def str_cmp(str1, str2):
    count1 = 0
    count2 = 0
    for i in range(len(str1)):
        if str1[i] >= "0" and str1[i] <= "9":
            count1 += 1

    for i in range(len(str2)):
        if str2[i] >= "0" and str2[i] <= "9":
            count2 += 1
    return count1 == count2

print (str_cmp("1234", "5672"))
print (str_cmp("1246", "Python"))
print (str_cmp("123Python", "Python123"))
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
True
False
True
```

Iterating String

Iterating over a string means accessing each of its characters simultaneously. There are various ways to iterate over the characters of a string in Python.

Example:

```
# Using simple iteration and range() function
str1 = "Programming"

for var in str1:
    print (var, end=' ')
print ("\n")


str2 = "Book"

for i in range(0, len(str2)):  # Iterate over index
    print (str2[i])
```

Output:

```
P r o g r a m m i n g
B
o
o
k
```

Example:

```
#Using enumerate() function
str3 = "Python"

# Iterate over the string
for i, v in enumerate(str3):
    print (v)
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
P
y
t
h
o
n
```

In Python, we can also iterate over the words of a string. A string of several words separated by spaces is given. In Python, many methods for iterating over words in a string exist.

split() Function

We can split the string into a list of words, but this method fails in a string containing punctuation marks. Let us take an example.

Example:

```
1.  str1 = "Python is a good prog., Python is easy to use, Python is easy to understand"
2.  print ("The original string is: " +  str1)
3.
4.  result = str1.split()   # using split() to extract words from string
5.
6.  print ("\nThe words of string are")
7.  for i in result:
8.      print (i)
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
The original string is: Python is a good prog., Python is easy to use, Python is easy to understand


The words of string are
Python
is
a
good
prog.,
Python
is
easy
to
use,
Python
is
easy
to
understand
```

re.findall() Function

This approach necessitates the use of regular expressions to complete the job. After filtering the string and extracting words while ignoring punctuation marks, the ***findall()*** function returns the list.

Example:

```
1.  #findall() example
2.  import re
3.
4.  str1= "Python is a good prog., Python is easy to use, Python is easy to understand."
5.  print ("The original string is: " +  str1)
6.
7.  result = re.findall(r'\w+', str1)
8.
9.  print ("\nThe words of string are")
10. for i in result:
11.     print (i)
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
The original string is: Python is a good prog., Python is easy to use, Python is easy to understand.
The words of string arePython
is
a
good
prog
Python
is
easy
to
use
Python
is
easy
to
understand
```

The String Module

The string module is a built-in module for using classes and constants. First, we have to import it before it can be used.

String Module Constants

There are various types of string constants defined in this module are:

string.ascii_letters

The following constants are concatenated: ascii_lowercase and ascii_uppercase.

string.ascii_lowercase

The lowercase letters 'abcdefghijklmnopqrstuvwxyz'. This value is not locale-dependent and will not change.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

string.ascii_uppercase

The uppercase letters 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'. This value is not locale-dependent and will not change.

string.digits

The string '0123456789'.

string.hexdigits

The string '0123456789abcdefABCDEF'.

string.punctuation

String of ASCII characters which are considered punctuation characters in the C locale: !”#$%&'()*+,-./:;<=>?@[\]^_`{|}~.

Example:

```
# import string library function
import string

# Storing the value in variable result variables
result1 = string.ascii_letters
result2 =string.ascii_lowercase
result3 =string.ascii_uppercase
result4 =string.digits
result5 =string.hexdigits
result6 =string.punctuation
# Printing the value
print (result1)
print (result2)
print (result3)
print (result4)
print (result5)
print (result6)
```

Output:

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789
0123456789abcdefABCDEF
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

String capwords() Function

This function splits the specified string into words and capitalizes each word.

Example:

```
# import string library function
import string
str= 'My name is james'
result= string.capwords(str)
print (result)
```

Output:

```
My Name Is James
```

String Module Classes

There are two types of string module classes:

1. Formatter
2. Template

Formatter

This class becomes beneficial if we wish to subclass it and provide the format string syntax. It is also identical to the **str.format()** function. Let's look at an example to grasp the Formatter class idea further.

Example:

```
from string import Formatter

formatter = Formatter()
print (formatter.format('{Faculty}', Faculty='Dr. John William'))
print (formatter.format('{} {Faculty}', 'Hello', Faculty='Dr. John William'))

print ('{} {Faculty}'.format('Hello', Faculty='Dr. John William')) # it is same as forma
        t()
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
Dr. John William
Hello Dr. John William
Hello Dr. John William
```

Template

We can design output specification syntax that is simpler to comprehend thanks to the Template class. The format generates placeholder names by combining the symbol **$** with Python identifiers that allow the use, such as alphanumeric letters and underscores. Let's look at an example together so that you can better grasp the concept behind the Template class.

Example:

```
# template example
from string import Template
# Creating a template
temp = Template('P is $P')
# Substitute value
print (temp.substitute({'P' : 1}))
# List of faculty stores the name and their id
Faculty = [('James William',11), ('John Smith',22), ('David Sam',33)]
temp = Template('Hello $name,   $id is your employee id')
        for var in Faculty:
            print (temp.substitute(name = var[0], id = var[1]))
```

Output:

```
P is 1
Hello James William,   11 is your employee id
Hello John Smith,   22 is your employee id
Hello David Sam,   33 is your employee id
```

Regular Expression

A regular expression is a unique string of characters that employs a particular syntax to assist you in fitting or locating other strings or sets of strings. This may be accomplished *via* the usage of a regular expression. Additionally, it establishes a pattern consisting of a group of strings that correlate to it. The functions at your disposal can make use of the **metacharacters** listed below in Table **4**.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Table 4 **Metacharacters description.**

\	It is used to drop the special meaning of a character.
[ ]	It represents a character class.
^	It matches the beginning.
$	It matches the end.
.	It matches any character except newline.
?	It matches zero or one occurrence.
|	It matches with any of the characters separated by it.
*	It shows any number of occurrences (including 0 occurrences)
+	It is for one or more occurrences.
{}	It indicates several occurrences of a preceding regular expression to match.
()	It encloses a group of regular expressions.

Example:

```
1.  import re
2.  p = re.compile('[a-m]')
3.  print (p.findall("My name is John Willian"))
```

Output:

```
['a', 'm', 'e', 'i', 'h', 'i', 'l', 'l', 'i', 'a']
```

The '\' metacharacter plays a crucial function in signaling different sequences. There are the following uses of metacharacter (\), which are shown in Table 5.

Table 5 **Metacharacters along with backslash (\).**

Symbol	Description
\d	It's equivalent to the set class [0-9] and matches every decimal digit.
\D	It can be used to match any non-digit character.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

\s	It is compatible with every whitespace character.
\S	It can be used to match any non-whitespace character.
\w	It is equivalent to the class [a-zA-Z0-9], which matches every alphanumeric character.
\W	It can be used to match any non-alphanumeric character.

Example:

```
1.  import re
2.  p1 = re.compile('\d')  # \d is equivalent to [0-9].
3.  print (p1.findall("My age is 40 year and my year of Birth is 1950"))
4.
5.  p2 = re.compile('\d+')  # \d+ will match a group on [0-9], group of one or greater size
6.  print (p2.findall("My age is 40 year and my year of Birth is 1950"))
7.
8.  p3 = re.compile('\w')   # \w is equivalent to [a-zA-Z0-9_].
9.  print (p3.findall("Welcome to the world of python. *"))
10.
11. p4 = re.compile('\w+')  # \w+ matches to group of alphanumeric character.
12. print (p4.findall("My age is 40 year and my year of Birth is 1950"))
13.
14. p5 = re.compile('\W')   # \W matches to non alphanumeric characters.
15. print (p5.findall("Welcome to the world of python. *"))
16.
17. p6 = re.compile('br*')  # '*' replaces the no. of occurrence of a character.
18. print (p6.findall("brbrbrbrbrbrbrbbbrrrrrr"))
```

Output:

```
['4', '0', '1', '9', '5', '0']

['40', '1950']

['W', 'e', 'l', 'c', 'o', 'm', 'e', 't', 'o', 't', 'h', 'e', 'w', 'o', 'r', 'l', 'd', 'o', 'f', 'p', 'y', 't', 'h', 'o', 'n']

['My', 'age', 'is', '40', 'year', 'and', 'my', 'year', 'of', 'Birth', 'is', '1950']

[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', '*']

['br', 'br', 'br', 'br', 'br', 'br', 'br', 'b', 'b', 'brrrrrr']
```

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
1.  import re
2.
3.  # Upon matching, 'y' is replaced by '!!' and CASE has been ignored
4.  print (re.sub('y', '~*' , 'My age is 36 year and my year of Birth is 1985', flags = re.IGNOR
        ECASE))
5.
6.  # Consider the Case Sensitivity, 'y' in "year", will not be reaplced.
7.  print (re.sub('y', '!!' , 'My age is 36 year and my year of Birth is 1985'))
8.
9.  # As count has been given value 1, the maximum times replacement occurs is 1
10. print (re.sub('y', '!!' , 'My age is 36 year and my year of Birth is 1985', count=1, flags = re
        .IGNORECASE))
11.
12. # 'r' before the patter denotes RE, \s is for start and end of a String.
13. print (re.sub(r'\sAND\s', ' & ', 'Book and note book', flags=re.IGNORECASE))
```

Output:

```
M~* age is 36 ~*ear and m~* ~*ear of Birth is 1985
M!! age is 36 !!ear and m!! !!ear of Birth is 1985
M!! age is 36 year and my year of Birth is 1985
Book & note book
```

LISTS

A list is an essential data type in Python, written as a comma-separated value enclosed with square brackets. There are the following important points about lists:

- List items are ordered, and that order will not change except for some list methods.
- The list allows duplicate values.
- The list is changeable, which means we can add and delete items in a list after it has been created.
- The first item has an index [0], and indexing will increase.

We can make any list by putting the series in square brackets [] and separating it with commas. It may contain any number of items of various kinds, such as integers, floats, strings, and so on. Let us take a basic example.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
# empty list
l1 = []
# list of integers
l2 = [4, 5, 6, 7]
# list with mixed data types
l3 = [2, "Python", "Book", 120.8]
print (l1)
print (l2)
print (l3)
```

Output:

```
[]
[4, 5, 6, 7]
[2, 'Python', 'Book', 120.8]
```

Every value in a list is indexed and can be accessed by referring to the index number. There are various ways to access a list of elements.

Using List Index

The index operator [] can gain access to a list object. The indices start at 0 in this system. Suppose there are seven items on the list; the order of the numbers then goes from 0 to 6. If you try to access values that are not contained in the index, you will receive an IndexError. One of the most important things to remember is that an index's value must always be an integer, and values of any other kind are not permitted. If you try to use a different data type for the index value, you will receive a TypeError message. Negative indexing is also allowed for list sequences. In negative indexing, -1 refers to the last item, -2 to the second last, -3 to the third last, and so on.

Example:

```
# List indexing
ListExample = ['P','Y','T','H','O','N']
print (ListExample [0])
print (ListExample [1])
print (ListExample [2])
print (ListExample [3])
print (ListExample [4])
#Negative index
print ('--Negative index--')
print (ListExample [-1])
print (ListExample [-2])
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Output:

```
P
Y
T
H
O
--Negative index--
N
O
```

List Operations

A slicing, deleting, removing, and clearing operation is performed on a list of items. By utilizing the slicing operator, it is possible to gain access to it across the range of items included in a list. The colon is used to symbolize it (:). The assignment operator (=) enables adding new elements to the list and modifying existing ones. The **delete()** method may be used to remove a particular object, while the **pop()** method can be used to remove an item from a certain index. We may use the **clear()** function to clean a list thoroughly. Let's look at an example to help better grasp how the mentioned operators and methods work.

Example:

```
1.  ml = ['p','y','t','h','o','n']
2.  print(ml[2:5])  #elements 3rd to 5th
3.  print(ml[:-6]) #elements beginning to 5th
4.  print(ml[3:]) #elements 4th to end
5.  print(ml[:]) #elements beginning to end
6.  del ml[2]  # delete one item
7.  print(ml)
8.  del ml[1:5] # delete multiple items
9.  print (ml)
10. mlist = ['P','Y','T','H','O','N']
11. mlist.remove('Y')
12. print (mlist)
13. print (mlist.pop(1))
14. print (mlist)
15. print (mlist.pop())
16. print (mlist)
17. mlist.clear()
18. print (mlist)
19. del ml   # delete entire list
20. print (ml)  # Error: List not defined
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
['t', 'h', 'o']
[]
['h', 'o', 'n']
['p', 'y', 't', 'h', 'o', 'n']
['p', 'y', 'h', 'o', 'n']
['p']
['P', 'T', 'H', 'O', 'N']
T
['P', 'H', 'O', 'N']
N
['P', 'H', 'O']
[]
Traceback (most recent call last):
  File "<string>", line 25, in <module>
NameError: name 'ml' is not defined
>
```

Updating Values in Lists

We can update single or multiple lists. It can use the (=) assignment operator to append and extend the method to accomplish the task. Let us take an example for understanding the mentioned operators and methods.

Example:

```
1.    # Add/Change List Elements
2.    var = [1, 2, 3, 4]
3.    var[0] = 9    # change the 1st item
4.    print (var)
5.
6.    var[1:4] = [22, 33, 44]  # change 2nd to 4th items
7.    print (var)
8.
9.    # Appending and Extending lists in Python using the append()
10.   var = [11, 22, 33]
11.   var.append(44)
12.   print (var)
13.
14.   var.extend([55, 66, 77])  #Add several items using extend() method
15.   print (var)
16.
17.   print (var + [224,353,464]) # Add + operator to combine two lists.
18.
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
[9, 2, 3, 4]
[9, 22, 33, 44]
[11, 22, 33, 44]
[11, 22, 33, 44, 55, 66, 77]
[11, 22, 33, 44, 55, 66, 77, 224, 353, 464]
```

Nested List

A nested list means a list within a list. Moreover, a list contains a sublist of elements. Suppose we have a list named **lst** that includes a list and sublist. A nested list is made up of a series of sublists separated by commas.

```
lst = ['ab', ['cd', ['efg', 'hij'], 'kl', 'mn'], 'o', 'p']
```

Example:

```
1.  lst = ['ab', ['cd', ['efg', 'hij'], 'kl', 'mn'], 'o', 'p']
2.
3.  print (L[2]) # Prints' o'
4.
5.  print (L[2][2]) # Prints' kl'
6.
7.  print (L[2][2][0]) # Prints 'k'
```

Output:

```
o
kl
k
```

It can add and remove items in a Nested list by using the *append(), insert(), extend(), pop(), del, and remove()* methods. It can also find length using *len()* and iterate through a nested list using for loop. Let us take an example to understand this concept.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
lst = ['P', ['ab', 'cd'], 'Y']
lst[1][1] = 0
print (lst)

lst[1].append('X')
print (lst)

lst[1].insert(0,'PP')
print (lst)

        lst[1].extend([77,88,99])
        print (lst)
```

Output:

```
['P', ['ab', 0], 'Y']
['P', ['ab', 0, 'X'], 'Y']
['P', ['PP', 'ab', 0, 'X'], 'Y']
['P', ['PP', 'ab', 0, 'X', 77, 88, 99], 'Y']
```

Aliasing

When we assign one variable to another, all of those variables will refer to the same object since variables are themselves objects. Additionally, the process of using two different IDs to refer to the same variable is known as aliasing. To better grasp this idea, let's look at a concrete illustration.

Example:

```
var1 = [555, 666,777]
var2 = var1     #Aliasing
print (var1 is var2)
print (var1)
print (var2)
```

Output:

```
[True
[555, 666, 777]
[555, 666, 777]
```

The same list has two different names in the above example, var1, and var2; however, after executing the assignment statement var2 = var1, you can see that var1 and var2 relate to the same list.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
var1 = [555, 666,777]
var2 = [555, 666,777]
print (var1)
print (var2)
print (var1 == var2)
print (var1 is var2)

var2 = var1
print (var1 == var2)
print (var1 is var2)

var2[0] = 999
print (var1)
print (var2)
```

Output:

```
[555, 666, 777]
[555, 666, 777]
True
False
True
True
[999, 666, 777]
[999, 666, 777]
```

Cloning Lists

If we want to change a list while keeping a copy of the original, we must copy the entire list, not just the reference. This method is often referred to as cloning to prevent the ambiguity of the word clone. The quickest and most straightforward way to clone a list is to use the slicing process.

Example:

```
list1 = [777, 888, 999]
list2 = list1[:]      # make a clone using slice
print (list1 == list2)
print (list1 is list2)
list2[0] = 555
print (list1)
print (list2)
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
True
False
[777, 888, 999]
[555, 888, 999]
```

There are various types of ways to clone a list.

Cloning List using Slicing

It is the most straightforward method of cloning a list. We use this form when we want to change a list while keeping a copy of the original. Let us take an example to understand this concept.

Example:

```
1.    def cloning_List(li1):
2.        lst1 = li1[:]   #Slice Operator
3.        return lst1
4.
5.
6.    lvar1 = [55,66,77,88,99]
7.    lvar2 = cloning_List(lvar1)
8.    print ("Original List:", lvar1)
9.    print ("After Cloning:", lvar2)
```

Output:

```
Original List: [55, 66, 77, 88, 99]
After Cloning: [55, 66, 77, 88, 99]
```

Cloning List using Extend()

The extend() function can be used to copy the lists into a new list. Let us take an example to understand this concept.

Example:

```
1.    def cloning_List(li1):
2.        lst1 = []
3.        lst1.extend(li1)
4.        return lst1
5.
6.    lvar1 = [55,66,77,88,99]
7.    lvar2 = cloning_List(lvar1)
8.    print ("Original List:", lvar1)
9.    print ("After Cloning:", lvar2)
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Output:

```
Original List: [55, 66, 77, 88, 99]
After Cloning: [55, 66, 77, 88, 99]
```

Cloning List using List()

A list can be cloned using a list() function. Let us take an example to understand this concept.

Example:

```
1.  def cloning_List(li1):
2.      lst1 = list(li1)
3.      return lst1
4.
5.  lvar1 = [55,66,77,88,99]
6.  lvar2 = cloning_List(lvar1)
7.  print ("Original List:", lvar1)
8.  print ("After Cloning:", lvar2)
```

Output:

```
Original List: [55, 66, 77, 88, 99]
After Cloning: [55, 66, 77, 88, 99]
```

Cloning List using List Comprehension

A list comprehension method is used to copy all the elements from one list to another. Let us take an example to understand this concept.

Example:

```
1.  def cloning_List(li1):
2.      lst1 = [i for i in li1]
3.      return lst1
4.
5.
6.  lvar1 = [55,66,77,88,99]
7.  lvar2 = cloning_List(lvar1)
8.  print ("Original List:", lvar1)
9.  print ("After Cloning:", lvar2)
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
Original List: [55, 66, 77, 88, 99]
After Cloning: [55, 66, 77, 88, 99]
```

Cloning List using Append()

The **append()** function can be used to append or add elements to the list or copy them to a new list. It's used to add elements to the list's last location. Let us take an example to understand this concept.

Example:

```
1.  def cloning_List(li1):
2.      lst1 = []
3.      for i in li1: lst1.append(i)
4.      return lst1
5.
6.
7.  lvar1 = [55,66,77,88,99]
8.  lvar2 = cloning_List(lvar1)
9.  print ("Original List:", lvar1)
10. print ("After Cloning:", lvar2)
```

Output:

```
Original List: [55, 66, 77, 88, 99]
After Cloning: [55, 66, 77, 88, 99]
```

Cloning List using Copy()

Use this method to copy all the elements from one list to another. Let us take an example to understand this concept.

Example:

```
1.  def cloning_List(li1):
2.      lst1 =[]
3.      lst1 = li1.copy()
4.      return lst1
5.
6.
7.  lvar1 = [55,66,77,88,99]
8.  lvar2 = cloning_List(lvar1)
9.  print ("Original List:", lvar1)
10. print ("After Cloning:", lvar2)
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
Original List: [55, 66, 77, 88, 99]
After Cloning: [55, 66, 77, 88, 99]
```

Built-in Functions

There are various types of built-in functions that are performed on the list. We can easily accomplish our tasks by directly using these functions (Table **6**).

Table 6 **List Built-in functions.**

cmp(list1, list2)	This function compares elements of both lists.
len(list)	This function returns the length of the list.
max(list)	This function returns an element with a max value from the list.
min(list)	This function returns an element from the list with a min value.
list(seq)	This function converts a tuple into a list.

Example:

```
1.  list1 = ['Amit', 10]
2.  list2 = ['David', 'James']
3.
4.  def cmp(p, q):
5.      return (p > q) - (p < q)
6.
7.  print (cmp(list1, list2))  # List comparison
8.
9.  print (len(list1))  # Length of list
10.
11.         list3 = [53535, 68868, 67656]
12.         print ("Maximum value element : ", max(list3))  # Getting Max value from list
13.
14.         print ("Minimum value element : ", min(list3)) # Getting Min value from list
```

Output:

```
-1
2
Maximum value element :  68868
Minimum value element :  53535
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

List Built-in List Methods

Some other built-in methods are also performed on the list. We can easily accomplish our task by directly using this method (Table **7**).

Table 7 **List built-in list methods.**

list.append(obj)	This method appends object obj to the list.
list.count(obj)	This method returns a count of how often obj occurs in the list.
list.extend(seq)	This method appends the contents of seq to the list.
list.index(obj)	
list.insert(index, obj)	This method inserts object obj into a list at offset index.
list.pop(obj=list[-1])	This method removes and returns the last object or obj from the list.
list.remove(obj)	This method removes object obj from the list.
list.reverse()	This method reverses objects of the list in place.
list.sort([func])	This method sorts objects of the list.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
1.lst1 = ['p', 'y', 't', 'h', 'o', 'n']
2.
3.lst1.append('programming')
4.print (lst1)
5.
6.lst1.extend(['x','y','z'])
7.print (lst1)
8.
9.print ("Count  : ", lst1.count('programming'))
10.
11.   print ("Index for programming : ", lst1.index( 'programming' ) )
12.
13.   lst1.insert( 3, 100)
14.   print (lst1)
15.
16.   print ("Poped List : ", lst1.pop())
17.
18.   lst1.remove('programming')
19.   print (lst1)
20.
21.   print (lst1.reverse())
22.
23.   lst2 = [44,464,686,313,668,242,575,1212,464]
24.
25.   lst2.sort()
26.   print (lst2)
27.
28.   print (lst2.reverse())
29.   print (lst2)
```

Output:

```
['p', 'y', 't', 'h', 'o', 'n', 'programming']
['p', 'y', 't', 'h', 'o', 'n', 'programming', 'x', 'y', 'z']
Count  :  1
Index for programming :  6
['p', 'y', 't', 100, 'h', 'o', 'n', 'programming', 'x', 'y', 'z']
Poped List :  z
['p', 'y', 't', 100, 'h', 'o', 'n', 'x', 'y']
None
[44, 242, 313, 464, 464, 575, 668, 686, 1212]
None
[1212, 686, 668, 575, 464, 464, 313, 242, 44]
```

List as Arrays

The list is a collection of elements of various data kinds, such as logical character values, numeric values, and so on. Because an array is used in many computer languages, including C, C++, Java, *etc.*, it is also well recognized. A group of items of homogenous make up an array. An exception

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

with the message “Incompatible data types” is raised if the array's items have different data types. Let's use an illustration to help you grasp this idea.

Example:

```
1.  # Simple list
2.  list1 = ['John','James', 10, 20, -5, 'David']
3.  print (list)
4.  # Using array in Python
5.  import array
6.  A = array.array('i', [30, 50, 80])
7.  # accessing elements of array
8.  for i in A:
9.      print (i)
```

Output:

```
<class 'list'>
30
50
80
```

There are differences between a list and an array in Python:

- A list can consist of elements belonging to different data types, but an array only consists of elements belonging to the same data type.
- A list does not need to import a module for declaration explicitly, but an array needs to import a module for representation explicitly.
- A list cannot directly handle arithmetic operations, but an array can directly manage arithmetic operations.
- A list can be nested to contain different elements, but an array must contain all nested elements of the same size.
- A list is preferred for a shorter sequence of data items, but an array is preferred for a longer sequence of data items.
- A modification is easier in the list compared to an array.
- The entire list can be printed without explicit looping but array using a loop to print or access the components.

Loops in Lists

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

When working with lists, “looping” involves displaying each item in the list in succession. In addition, the Python operation is an iteration through a list. Python provides a wide variety of options for iterating through a list in a program.

For Loop in the List

Displaying each item on a list may be accomplished with the help of a for loop. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  list = [11, 22, 33, 44, 55]
2.
3.  for var in list:
4.      print (var)
```

Output:

```
11
22
33
44
55
```

Let's say we wish to utilize the standard for loop, which iterates through the element until the range we provide. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  list = [11, 22, 33, 44, 55]
2.
3.  length = len(list) # getting the length of a list
4.
5.  for var in range(length):
6.      print (list[var])
```

Output:

```
11
22
33
44
55
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

While Looping in the List

Although it operates in a manner comparable to a for loop, its syntax is distinct. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  list = [11, 22, 33, 44, 55]
2.
3.  length = len(list) # getting length of list
4.  x1 = 0
5.  while x1 < length:
6.      print (list[x1])
7.      X1 += 1
```

Output:

```
11
22
33
44
55
```

List Comprehension

In Python, it is necessary to use it to iterate through a list. It is an alternative method for constructing a list of components with a particular quality. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  list = [11, 22, 33, 44, 55]
2.
3.  [print (i) for i in list]   # Using list comprehension
```

Output:

```
11
22
33
44
55
```

Enumerate() Method

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

This method can convert the list into an iterable list of tuples. Let us take an example to understand this concept.

Example:

```
list = [11, 22, 33, 44, 55]

for i, val in enumerate(list):   # Using enumerate()
    print (i, "=",val)
```

Output:

```
0 = 11
1 = 22
2 = 33
3 = 44
4 = 55
```

Numpy in Loop

When working with extensive n-dimensional lists, it is often preferable to use an external library such as NumPy. To better grasp this idea, let's look at a concrete illustration.

Example:

```
import numpy as np

var = np.arange(4)
var = var.reshape(2, 2)

for var1 in np.nditer(var): # iteration
    print (var1)
```

Output:

```
0
1
2
3
```

List in Functions

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

When you provide a list to a function, you pass a list variable to the function. The value of the list variable is then copied to the function arguments by the function. By using the function's arguments, it is, therefore quite simple to display the list's values. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  def students(f):
2.      for i in f:
3.        print (i)
4.  l1 = ["David", "James", "William", "John"]
5.  students(l1)
```

Output:

```
David
James
William
John
```

A function can return a list. When you say “returning list to function,” you are telling the function that was called to get the list's value and catch. To print the results, another variable has assigned a calling function. To better grasp this idea, let's look at a concrete illustration.

Example:

```
1.  def students(f):
2.      for i in f:
3.        return (i)
4.  l1 = ["David", "James", "William", "John"]
5.  students(l1)
6.
7.  for i in l1:
8.      students(i)
9.      print (i)
10.
```

Output:

```
David
James
William
John
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

CONCLUDING REMARKS

In this chapter, we spoke about strings and lists with their different operations, such as concatenating, appending, multiplying, and other things. The string formatting operator and the process that it uses is the focus of this section. The built-in string methods slice, ord(), and chr(), comparing and iterating a string, and further clarifying regular expression with an excellent example are also covered in this section. In the following chapter, we will move on to the next topic utilized in Python: tuples and dictionaries.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:39:17.